

PROJECT CONCEPT NOTE

AI Email Copilot using Retrieval-Augmented Generation (RAG)

Application Name: **AI Email Copilot**

Domain: Artificial Intelligence | Generative AI | Cloud Computing

Submitted To: IBM SkillsBuild / Project Evaluation Committee

Prepared By: **Team GenCrew**

1. Project Title

- AI Email Copilot using Retrieval-Augmented Generation (RAG)

2. Application Name

- AI Email Copilot

3. Problem Statement

- Writing professional emails is time-consuming and challenging for students, job seekers, and professionals.
- Most existing AI tools rely only on prompt engineering, producing generic and contextually shallow emails.
- No retrieval mechanism means the model lacks grounding in professional communication norms.
- Users struggle with appropriate tone, structure, and personalization when composing formal emails.
- This project solves the problem by using RAG to retrieve relevant examples before generation.

4. Project Objective

- Design and deploy an AI-powered web app that generates personalized, professional emails using RAG.
- Retrieve semantically relevant email templates from a FAISS vector database using HuggingFace embeddings.
- Use Google Gemini 2.5 Flash Lite as the LLM to produce structured, context-aware email content.
- Demonstrate practical integration of LangChain, FAISS, HuggingFace, and Gemini in a production app.
- Deploy the solution on AWS App Runner as a containerized, scalable cloud service using Docker.

5. Target Users

- Undergraduate and postgraduate students
- Job seekers and fresh graduates
- Working professionals
- HR executives and recruiters
- Freelancers
- Sales and business communication teams

6. Proposed Solution

- Implements the Retrieval-Augmented Generation (RAG) architecture for grounded email generation.
- Curated email templates (5 categories) are stored as dense vectors in a FAISS vector database.
- User query is encoded using HuggingFace all-MiniLM-L6-v2 and matched against the index.
- Top-K retrieved templates are injected into a structured LangChain prompt for Gemini 2.5 Flash Lite.
- Streamlit provides the user-facing interface; FastAPI exposes a REST API for programmatic access.
- Fully containerized with Docker and deployed on AWS App Runner for production-grade cloud hosting.

7. Key Features

- AI-powered professional email generation
- RAG pipeline with FAISS semantic search
- 5 email purposes (cold email, internship, follow-up, apology, sales)
- 5 communication tones + 3 length options
- Structured prompt engineering via LangChain
- Responsive Streamlit UI
- FastAPI REST backend
- Secure API key management (.env / AWS env vars)
- Docker containerization
- AWS App Runner cloud deployment

8. Technologies Used

Component	Technology
Programming Language	Python 3.11
Frontend / UI	Streamlit
Backend API	FastAPI + Uvicorn
AI Orchestration	LangChain
Large Language Model	Google Gemini 2.5 Flash Lite
Embedding Model	HuggingFace all-MiniLM-L6-v2
Vector Database	FAISS (Facebook AI Similarity Search)
Env Management	python-dotenv
Containerization	Docker
Cloud Deployment	AWS App Runner

9. LLM Model Used

- **Google Gemini 2.5 Flash Lite** — primary generative model; low-latency, high instruction-following capability.
- Accessed via **google-generativeai** SDK and integrated through **langchain-google-genai** package.
- Configured with **temperature = 0.7** to balance creativity with coherent, structured output.
- **HuggingFace all-MiniLM-L6-v2** — embedding model producing 384-dimensional sentence vectors.
- Embeddings generated via **HuggingFace Inference Endpoint API** — no local PyTorch download needed.
- Embedding API choice reduces Docker image size significantly, improving cloud cold-start performance.

10. Application Workflow

Stage 1 — User Input	User provides sender, recipient, company, purpose, tone, length, and key points via Streamlit UI.
Stage 2 — Query Embedding	Inputs combined into a semantic query string and encoded using HuggingFace all-MiniLM-L6-v2.
Stage 3 — Semantic Search	Encoded vector is matched against the FAISS index; top-K relevant email templates are retrieved.
Stage 4 — Prompt Engineering	Retrieved templates + user inputs assembled into a structured LangChain prompt with format rules.
Stage 5 — Email Generation	Prompt sent to Google Gemini 2.5 Flash Lite; model generates a complete, formatted email.
Stage 6 — Output Delivery	Final email with subject line, salutation, body, and closing displayed in the Streamlit interface.

11. Expected User Experience

- Clean, form-based Streamlit interface — no technical knowledge required.
- User fills sender/recipient/purpose/tone/length/key points and clicks Generate.
- System retrieves templates and generates a complete, professional email within seconds.
- Output includes subject line, salutation, structured body, and professional closing.
- Generated email can be directly copied and sent with minimal or no editing.

12. Expected Outcomes

- Significant reduction in time and effort required to compose professional emails.
- Contextually accurate, well-structured emails matching purpose, tone, and user key points.
- Practical demonstration of a full RAG pipeline using FAISS, HuggingFace, and Gemini.
- Showcase of end-to-end MLOps: data prep → model integration → UI → Docker → cloud deployment.
- Scalable, production-grade AI application deployed on AWS App Runner.

13. Innovation and Uniqueness

- Applies RAG to professional email generation — a domain typically limited to prompt-only approaches.
- Retrieval grounds the LLM output in real template examples, producing more contextually accurate emails.
- Uses HuggingFace Inference Endpoint API instead of a local model — reduces Docker image by ~2 GB.
- Parallel REST API (FastAPI) alongside Streamlit UI demonstrates production-grade architecture.
- Full containerization and AWS App Runner deployment distinguishes it from typical student AI projects.

14. Future Scope

- Expand the knowledge base with additional email categories (vendor, performance review, support).
- Add user feedback loop to improve retrieval quality through reinforcement signals.
- Introduce multi-language support for regional professional email generation.
- Integrate Gmail / Outlook API for one-click email dispatch directly from the application.
- Add user authentication, email history, and session management for personalized experience.

15. Conclusion

- AI Email Copilot demonstrates a practical, production-ready application of Retrieval-Augmented Generation.
- Combines FAISS semantic search, HuggingFace embeddings, and Google Gemini 2.5 Flash Lite via LangChain.
- Delivers context-aware, personalized email generation that surpasses prompt-only approaches in quality.
- Meets industry standards: modular codebase, secure key management, Docker, and AWS cloud deployment.
- Represents a holistic implementation of the modern AI development lifecycle from data to cloud.